

FEMM_to_LTspice Jupyter Notebook

Project FEMM_to_LTspice (project number 7801) The Power Electronics and Renewable Energy Lab (PEARL) The Andrew and Enra Viterbi Faculty of Electrical and Computer Engineering Technion - Israel Institute of Technology Author: Shahar Zamir Last edited: 14 August 2024 Last edited by: Shahar Zamir

```
In [1]: import numpy as np
import matplotlib.pyplot as plt

import femm

import ltspice
import xml.etree.ElementTree as ET

import os
import datetime
import subprocess
```

FEMM

```
In [2]: # FEMM

# Path to the FEMM installation directory
# femm_path = r'C:\femm42' # Modify this path to match your FEMM installation dire
femm_path = r'C:\Program Files\femm42' # Modify this path to match your FEMM insta

# Append the FEMM path to the PATH environment variable
os.environ['PATH'] += os.pathsep + femm_path

# Verify the PATH update
# print(os.environ['PATH'])
```

```
In [3]: # At this point you can choose:
# 1 - to load an exciting model of FEMM (skip down to the right cell)
# 2 - or Create a new model in FEMM (stay here and continue)

# Open FEMM and create a new magnetostatic problem
femm.openfemm()
femm.newdocument(0) # 0 for magnetic problem

# Define problem
# mi_probdef(freq[Hz],units,type,precision,depth,minangle,(acsolver))
# DC - for faster results (Debug - toggle comment to switch AC/DC):

# femm.mi_probdef(0, "millimeters", "planar", 1e-8, 12.7, 30) # DC freq=0
femm.mi_probdef(10e+3, "millimeters", "planar", 1e-8, 12.7, 30) # AC 10kHz

# Minimize FEMM GUI manually
```

```
In [4]: # Toroid Inductor Geometry
```

```

# Core dimensions
D_in = 12.7      # Inner diameter in mm
D_out = 20.32    # Outer diameter in mm
h = 12.7         # Height in mm (actually set in *depth* while defining the problem)
r_in = D_in / 2  # Inner radius
r_out = D_out / 2 # Outer radius
num_of_turns = 20 # Number of coil turns

# Coil dimensions
coil_diameter = 2.588 # Diameter of 10 AWG wire (approximately 2.588 mm)
# Gap for coil dimensions (adjust as needed)
# gap_for_coil = coil_diameter + 0.1
gap_for_coil = 0.5 # smaller than whats needed just for better visibility - shouldnt

```

```

In [5]: # Define materials
femm.mi_getmaterial('Air')
femm.mi_getmaterial('10 AWG')

# Add a new non-linear material defined by B-H curve (data points)
# Read the B-H curve data from a .dat file
data_Ferrite_N87_BH_Curve = np.loadtxt('Ferrite_N87_BH_Curve.dat')
B_values = data_Ferrite_N87_BH_Curve[:, 0] # First column (B values)
H_values = data_Ferrite_N87_BH_Curve[:, 1] # Second column (H values)
# Optional - Assain manually (But May NOT Work):
# B_values = [0, 0.2, 0.4, 0.6, 0.8, 1.0] # T
# H_values = [0, 1000, 2000, 3000, 4000, 5000] # A/m

# Define the non-linear material with the B-H curve
material_name = "Ferrite_N87"
femm.mi_addmaterial(material_name,
                    1, # Relative permeability in the x direction (op
                    1, # Relative permeability in the y direction (op
                    0.35, # H_c Coercivity (S/m)
                    0, # Source Current Density - J
                    0.0001, # Electrical Conductivity - Sigma
                    0, # Lam_thickness
                    0, # Lam_fill
                    1) # Lam_type (default value)

# Add B-H curve data points
for B, H in zip(B_values, H_values):
    femm.mi_addbhpnt(material_name, B, H)

# Add circuits (define coils and current value. +/- for current direction. Value in
femm.mi_addcircprop("inner_coil", 5, 1)
femm.mi_addcircprop("outter_coil", -5, 1)

```

```

In [6]: # Drawing

# Core center
center_x = 0
center_y = 0

# Create core geometry (toroid cross-section)
# Inner

```

```
femm.mi_drawarc(center_x + r_in, center_y, center_x - r_in, center_y, 180, 1)
femm.mi_drawarc(center_x - r_in, center_y, center_x + r_in, center_y, 180, 1)
# Add gaps from inside of the core
femm.mi_drawarc(center_x + r_in - gap_for_coil, center_y, center_x - r_in + gap_for
femm.mi_drawarc(center_x - r_in + gap_for_coil, center_y, center_x + r_in - gap_for
# Outer
femm.mi_drawarc(center_x + r_out, center_y, center_x - r_out, center_y, 180, 1)
femm.mi_drawarc(center_x - r_out, center_y, center_x + r_out, center_y, 180, 1)
# Add gaps from outside of the core
femm.mi_drawarc(center_x + r_out + gap_for_coil, center_y, center_x - r_out - gap_f
femm.mi_drawarc(center_x - r_out - gap_for_coil, center_y, center_x + r_out + gap_f

# At this point you should see drawings in the GUI, open it to make sure.
```

```
In [7]: # Assign materials
# Outer materail: Outside r_out + gap_for_coil
femm.mi_addblocklabel(center_x + r_out + gap_for_coil + 1, center_y + 5)
femm.mi_selectlabel(center_x + r_out + gap_for_coil + 1, center_y + 5)
femm.mi_setblockprop('Air', 1, 0, '', 0, 0, 0)
femm.mi_clearselected()

# Inner Material: Inside r_in - gap_for_coil
femm.mi_addblocklabel(center_x + 1, center_y + 1)
femm.mi_selectlabel(center_x + 1, center_y + 1)
femm.mi_setblockprop('Air', 1, 0, '', 0, 0, 0)
femm.mi_clearselected()

# Core Material: Between r_out and r_in
femm.mi_addblocklabel(center_x + (r_in + r_out) / 2, center_y)
femm.mi_selectlabel(center_x + (r_in + r_out) / 2, center_y)
femm.mi_setblockprop('Ferrite_N87', 1, 0, '', 0, 0, 0)
femm.mi_clearselected()
```

```
In [8]: # Assign circuits (coil)
# Outer Circuit: Between r_out and r_out + gap_for_coil
femm.mi_addblocklabel(center_x + r_out + gap_for_coil / 2, center_y)
femm.mi_selectlabel(center_x + r_out + gap_for_coil / 2, center_y)
femm.mi_setblockprop("10 AWG", 1, 0, "outer_coil", 0, 0, 20)
femm.mi_clearselected()

# Inner Circuit: Between r_in - gap_for_coil and r_in
femm.mi_addblocklabel(center_x + r_in - gap_for_coil / 2, center_y)
femm.mi_selectlabel(center_x + r_in - gap_for_coil / 2, center_y)
femm.mi_setblockprop("10 AWG", 1, 0, "inner_coil", 0, 0, 20)
femm.mi_clearselected()
```

```
In [9]: # Create IABC open boundary (auto optin to creat boundry condition)
femm.mi_makeABC()
```

```
In [10]: # Save the document - optional - save with date and time to avoid overriding

# Get the current date and time
now = datetime.datetime.now()
# Format the date and time as a string
timestamp = now.strftime("%d%m%Y_%H%M")
```

```

# Create the filename with the timestamp
filename = f'toroid_core_geometry_{timestamp}.fem'

# Save the document with the new filename
femm.mi_saveas(filename)

# Save the document with the name you want
# femm.mi_saveas('toroid_core_geometry_with_the_name_I_want.fem')

```

```

In [11]: # Analyze the model. This process may take some time, particularly when using frequ
# Wait until the [*] indicator on the left changes to a number.
femm.mi_analyze()
femm.mi_loadsolution()

```

```

In [12]: # Calculate the inductancy L

# Get the circuit properties
circuit_name = 'inner_coil' # Replace with the actual circuit name
circuit_properties = femm.mo_getcircuitproperties(circuit_name)
print(circuit_properties)

current = circuit_properties[0]
flux_linkage = circuit_properties[2]

# Calculate the inductance
L = flux_linkage / current
print(f'Inductance: {L} H')

# Extract the real part and convert to microhenries
L_microhenry = L.real * 1e6
print(f'Inductance: {L_microhenry:.2f} μH')

# Format L_microhenry to 2 decimal places and assign to new_L_microhenry
new_L_microhenry = round(L_microhenry, 2)
# print(new_L_microhenry)

```

```

[5, (0.4032410495956466+39.83361252847401j), (0.0006339716335113877-6.11411785499655
7e-06j)]

```

```

Inductance: (0.00012679432670227754-1.2228235709993114e-06j) H

```

```

Inductance: 126.79 μH

```

```

In [13]: # Calculate the ESR (Equivalent Series Resistance) of the inductor

# Define the area for integration, where the coil goes
# This part depends on the specific model and how it is defined
femm.mo_seteditmode('area')
femm.mo_selectblock(center_x + r_out + gap_for_coil / 2, center_y) # Replace with
femm.mo_selectblock(center_x + r_in - gap_for_coil / 2, center_y)

# Integrate the resistive losses
power_dissipation = femm.mo_blockintegral(4) # Integral type 4 is for resistive lo
print(f'Power Dissipation: {power_dissipation} Watt')

# Calculate the ESR
ESR = power_dissipation / (current ** 2)
print(f'Equivalent Series Resistance (ESR): {ESR} Ω')

```

```
# convert to mili ohm
ESR_mili = ESR.real * 1e3
print(f'ESR: {ESR_mili:.4f} mΩ')

# Format ESR_mili to 2 decimal places and assign to new_ESR_mili
new_ESR_mili = round(ESR_mili, 2)
# print(new_ESR_mili)
```

Power Dissipation: 0.03904977484928465 Watt
 Equivalent Series Resistance (ESR): 0.0015619909939713862 Ω
 ESR: 1.5620 mΩ

```
In [14]: # Close FEMM
femm.closefemm()
```

If you used the above method, you shell now skip "FEMM - use an exiting file" and proceed to the "LTspice" part down below.

FEMM - Use an existing file

```
In [ ]: # Use an existing file
# CHANGE PATH

femm.openfemm()
# Open an existing FEMM file
fem_file_path = r'C:\Users\xzamirt_lenovo\anaconda3\project_femm_to_spice\toroid_co
femm.opendocument(fem_file_path)
ans_file_path = r'C:\Users\xzamirt_lenovo\anaconda3\project_femm_to_spice\toroid_co
femm.opendocument(ans_file_path)

# Analyze the model
# Toggle comment to analyze - may take some time
# femm.mi_analyze()

femm.mi_loadsolution()

# Minimize FEMM GUI manually
```

```
In [ ]: # Toroid Inductor Geometry - should be changed acording to your design

# Core dimensions
D_in = 12.7 # Inner diameter in mm
D_out = 20.32 # Outer diameter in mm
h = 12.7 # Height in mm (actually set in *depth* while defining the problem
r_in = D_in / 2 # Inner radius
r_out = D_out / 2 # Outer radius
num_of_turns = 20 # Number of coil turns
```

```

# Coil dimensions
coil_diameter = 2.588 # Diameter of 10 AWG wire (approximately 2.588 mm)
# Gap for coil dimensions (adjust as needed)
# gap_for_coil = coil_diameter + 0.1
gap_for_coil = 0.5 # smaller than what's needed just for better visibility - shouldn't

# Core center
center_x = 0
center_y = 0

```

```

In [ ]: # Calculate the inductance L

# Get the circuit properties
circuit_name = 'inner_coil' # Replace with the actual circuit name
circuit_properties = femm.mo_getcircuitproperties(circuit_name)
print(circuit_properties)

current = circuit_properties[0]
flux_linkage = circuit_properties[2]

# Calculate the inductance
L = flux_linkage / current
print(f'Inductance: {L} H')

# Extract the real part and convert to microhenries
L_microhenry = L.real * 1e6
print(f'Inductance: {L_microhenry:.2f} μH')

# Format L_microhenry to 2 decimal places and assign to new_L_microhenry
new_L_microhenry = round(L_microhenry, 2)
# print(new_L_microhenry)

```

```

In [ ]: # Calculate the ESR (Equivalent Series Resistance) of the inductor

# Define the area for integration, where the coil goes
# This part depends on the specific model and how it is defined
femm.mo_seteditmode('area')
femm.mo_selectblock(center_x + r_out + gap_for_coil / 2, center_y) # Replace with
femm.mo_selectblock(center_x + r_in - gap_for_coil / 2, center_y)

# Integrate the resistive losses
power_dissipation = femm.mo_blockintegral(4) # Integral type 4 is for resistive loss
print(f'Power Dissipation: {power_dissipation} Watt')

# Calculate the ESR
ESR = power_dissipation / (current ** 2)
print(f'Equivalent Series Resistance (ESR): {ESR} Ω')

# convert to milli ohm
ESR_mili = ESR.real * 1e3
print(f'ESR: {ESR_mili:.4f} mΩ')

# Format ESR_mili to 2 decimal places and assign to new_ESR_mili
new_ESR_mili = round(ESR_mili, 2)
# print(new_ESR_mili)

```

```
In [ ]: # Close FEMM
femm.closefemm()
```

LTspice

```
In [15]: # Path to the LTspice installation directory
ltspice_path = r'C:\Program Files\ADI\LTspice'

# Append the LTspice path to the PATH environment variable
os.environ['PATH'] += os.pathsep + ltspice_path

# Verify the PATH update
# print(os.environ['PATH'])
```

```
In [16]: def open_ltspice_gui(ltspice_exe, sim_file):
# Command to open LTspice GUI and load the specified file
command = f'start "" "{ltspice_exe}" "{sim_file}"'
print(f"Executing command: {command}")

try:
    subprocess.run(command, shell=True)
    print("LTspice GUI opened and file loaded.")
except subprocess.CalledProcessError as e:
    print(f"An error occurred while running the simulation: {e}")
```

```
In [17]: # Path to LTspice executable
ltspice_exe = 'LTspice.exe'

# Path to your LTspice file (.asc or .net)
sim_file = r'C:\Users\xzamirt_lenovo\anaconda3\project_femm_to_spice\Boost_Converte
```

```
In [18]: # Call the function to open LTspice and load the file
open_ltspice_gui(ltspice_exe, sim_file)
```

```
Executing command: start "" "LTspice.exe" "C:\Users\xzamirt_lenovo\anaconda3\project_femm_to_spice\Boost_Converter_V0.asc"
LTspice GUI opened and file loaded.
```

```
In [19]: # Important Note:
# At this stage, you need to generate the .raw file. There are two options:
# 1. Manually run the simulation through the GUI to generate the .raw file.
# 2. Execute the next cell, then CLOSE the GUI manually (you can safely ignore any
```

```
In [20]: # Define the command to run LTspice with the simulation
command = [ltspice_exe, '-Run', sim_file]

# Run the command
result = subprocess.run(command, capture_output=True, text=True)

# TODO "stop" the cell
# for now close the GUI manually before proceeding
```

```
In [21]: def load_simulation_results(sim_file):
    raw_file = sim_file.replace('.asc', '.raw')

    # if not os.path.exists(raw_file):
    #     raise FileNotFoundError(f"The raw file '{raw_file}' does not exist.")

    l = ltspice.Ltspice(raw_file)
    l.parse()
    return l
```

```
In [22]: # Load the simulation results
ltspice_data = load_simulation_results(sim_file)
```

```
In [23]: # Get the list of available signals

signals = ltspice_data.variables
# getData()
print("Available signals:", signals)
```

Available signals: ['time', 'V(v_in)', 'V(n002)', 'V(v_out)', 'V(n001)', 'V(vst)', 'V(n5v)', 'V(p5v)', 'V(vc)', 'V(d)', 'V(n003)', 'I(B1)', 'Id(M2)', 'Ig(M2)', 'Is(M2)', 'I(C1)', 'I(C2)', 'I(D1)', 'I(L1)', 'I(R1)', 'I(Esr)', 'I(V1)', 'I(V2)', 'I(V3)', 'I(V4)', 'I(V5)', 'Ix(u2:IN+)', 'Ix(u2:IN-)', 'Ix(u2:POS)', 'Ix(u2:NEG)', 'Ix(u2:OUT)']

```
In [24]: # Path to your LTspice schematic file (.asc)
asc_file = r'C:\Users\xzamirt_lenovo\anaconda3\project_femm_to_spice\Boost_Converte

# Read the .asc file with a specific encoding
with open(asc_file, 'r', encoding='latin-1') as file:
    lines = file.readlines()

# Print the first few lines to understand the format
# for line in lines[:10]:
#     print(line.strip())
```

```
In [25]: # Function to modify the inductance value of an inductor

def modify_inductance(lines, inductor_name, new_value):
    for i, line in enumerate(lines):
        if line.strip() == f"SYMATTR InstName {inductor_name}":
            if lines[i + 1].strip().startswith("SYMATTR Value"):
                # Print the original lines for verification
                print(f"Original line {i}: {lines[i].strip()}")
                print(f"Original line {i + 1}: {lines[i + 1].strip()}")

                # Modify the inductance value
                lines[i + 1] = f"SYMATTR Value {new_value}\n"

                # Print the modified lines for verification
                print(f"Modified line {i}: {lines[i].strip()}")
                print(f"Modified line {i + 1}: {lines[i + 1].strip()}")
                break
    return lines
```

```
In [26]: # Modify the inductance of L1 to a new value (e.g., 200uH)

# new_inductance_value = '200u'
new_inductance_value = f"{new_L_microhenry}u"

lines = modify_inductance(lines, 'L1', new_inductance_value)

# Save the modified .asc file
with open(asc_file, 'w', encoding='latin-1') as file:
    file.writelines(lines)

print("Inductance value modified successfully.")

# To see the change reload the .asc file
```

Original line 77: SYMATTR InstName L1
 Original line 78: SYMATTR Value 126.79u
 Modified line 77: SYMATTR InstName L1
 Modified line 78: SYMATTR Value 126.79u
 Inductance value modified successfully.

```
In [27]: # Function to modify the ESR value

def modify_ESR(lines, ESR_name, new_value):
    for i, line in enumerate(lines):
        if line.strip() == f"SYMATTR InstName {ESR_name}":
            if lines[i + 1].strip().startswith("SYMATTR Value"):
                # Print the original lines for verification
                print(f"Original line {i}: {lines[i].strip()}")
                print(f"Original line {i + 1}: {lines[i + 1].strip()}")

                # Modify the inductance value
                lines[i + 1] = f"SYMATTR Value {new_value}\n"

                # Print the modified lines for verification
                print(f"Modified line {i}: {lines[i].strip()}")
                print(f"Modified line {i + 1}: {lines[i + 1].strip()}")
                break
    return lines
```

```
In [28]: # Modify the resistance of ESR to a new value (e.g., 1.5m)

# new_ESR_value = '1.55m'
new_ESR_value = f"{new_ESR_mili}m"

lines = modify_ESR(lines, 'ESR', new_ESR_value)

# Save the modified .asc file
with open(asc_file, 'w', encoding='latin-1') as file:
    file.writelines(lines)

print("ESR value modified successfully.")
```

```
# To see the change reload the .asc file
```

Original line 113: SYMATTR InstName ESR
Original line 114: SYMATTR Value 1.56m
Modified line 113: SYMATTR InstName ESR
Modified line 114: SYMATTR Value 1.56m
ESR value modified successfully.

Close and Open LTspice GUI to confirm the changes. Then run the simulation and watch the results.

```
In [29]: # Call the function to open LTspice and Load the file  
open_ltspice_gui(ltspice_exe, sim_file)
```

Executing command: start "" "LTspice.exe" "C:\Users\xzamirt_lenovo\anaconda3\project_femm_to_spice\Boost_Converter_V0.asc"
LTspice GUI opened and file loaded.

```
In [30]: # Run the command  
result = subprocess.run(command, capture_output=True, text=True)
```

```
In [31]: # Load the simulation results  
ltspice_data = load_simulation_results(sim_file)
```

```
In [32]: # Optional: Plot the results directly in the notebook.
```

```
# Access specific signals  
if 'V(v_in)' in signals:  
    voltage_in = ltspice_data.getData('V(v_in)')  
    time = ltspice_data.getTime()  
  
if 'V(v_out)' in signals:  
    voltage_out = ltspice_data.getData('V(v_out)')  
  
# Plot both signals on the same plot  
plt.plot(time, voltage_in, label='V(v_in)')  
plt.plot(time, voltage_out, label='V(v_out)')  
  
# Add labels and title  
plt.xlabel('Time (s)')  
plt.ylabel('Voltage (V)')  
plt.title('Basic plot - Voltage vs. Time')  
  
# Add grid and Legend  
plt.grid(True)  
plt.legend()  
  
# Save the plot to a file  
plt.savefig('Basic.png')  
  
# Show the plot
```

```
plt.show()

else:
    print("Signal 'V(v_out)' not found in the simulation results.")
```

```
C:\Users\xzamirt_lenovo\AppData\Local\Temp\ipykernel_7904\3843337959.py:5: Deprecati
onWarning: Call to deprecated method getData. (use method which follows pep8) -- Dep
recated since version 1.0.0.
```

```
voltage_in = ltspice_data.getData('V(v_in)')
```

```
C:\Users\xzamirt_lenovo\AppData\Local\Temp\ipykernel_7904\3843337959.py:6: Deprecati
onWarning: Call to deprecated method getTime. (use method which follows pep8) -- Dep
recated since version 1.0.0.
```

```
time = ltspice_data.getTime()
```

```
C:\Users\xzamirt_lenovo\AppData\Local\Temp\ipykernel_7904\3843337959.py:9: Deprecati
onWarning: Call to deprecated method getData. (use method which follows pep8) -- Dep
recated since version 1.0.0.
```

```
voltage_out = ltspice_data.getData('V(v_out)')
```

